SIEMENS

Functions on devices 1

Functions on device items 2

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

NOTICE

This document was downloaded from the following site:

https://docs.industrial-operations-x.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 Functions on devices ... 6

1.1 Mandatory attributes of devices ... 6
1.2 Get type identifier of devices and device items ... 7
1.3 Set App ID on device and device Items ... 11
1.4 Get App ID on device and device Items ... 12
1.5 Remove App ID on device and device items ... 13
1.6 Creating a device ... 14
1.7 Enumerating devices ... 16
1.8 Accessing devices ... 20
1.9 Accessing the TIA Portal hardware catalog ... 23
1.10 Deleting a device ... 27
1.11 Bulk changing hardware parameters ... 27

2 Functions on device items ... 30

2.1 Mandatory attributes of device items ... 30
2.2 Creating and plugging a device item ... 33
2.3 Moving device items into another slot ... 37
2.4 Copying a device item ... 39
2.5 Setting user defined logo settings ... 40
2.6 Deleting a device item ... 41
2.7 Enumerate device items ... 42
2.8 Accessing device items ... 46
2.9 Accessing device item as interface ... 52
2.10 Accessing change device ... 53
2.11 Accessing attributes of an I/O device interface ... 55
2.12 Getting subnet of device item ... 57
2.13 Accessing attributes of IoController ... 58
2.14 Accessing attributes of IoConnector ... 59
2.15 Accessing address controller ... 62
2.16 Accessing addresses ... 63
2.17 Accessing hardware identifiers ... 66
2.18 Accessing hardware identifier controller ... 67
2.19 Accessing channels of device items ... 68
2.20 Accessing normalized type identifiers ... 70
2.21 Creating and exporting psc file ... 70
2.22 Connection handling for extension racks ... 73
2.23 Telecontrol management datapoint ... 74

Functions on devices

1

1.1 Mandatory attributes of devices

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

Every device or device item provides certain mandatory attributes which can be read and/or written. These attributes are always the same as in the TIA Portal user interface.

The following attributes are supported in Openness:

Attribute name	Data type	Writeable	Access	Comment
Author	string	read/write	dynamic	
Comment	string	read/write	dynamic	sometimes only read access
CommentML	MultilingualTextItem	read/write	dynamic	sometimes only read access
IsGsd	bool	read		TRUE, if the device description is installed via GSD/GSDML
Name	string	read/write		sometimes only read access
TypeIdentifier	string	read		
TypeName	string	read	dynamic	

Program code: Mandatory attributes of a device

Modify the following program code to get the mandatory attributes of a device:



```
Copies the following program code to clipboard.

private static void AccessingAttributesOfDevice(Device
device)
{
   string nameValue = device.Name;
   bool isGsdValue = device.IsGsd;
}
```

Program code: Mandatory attributes with dynamic access

Modify the following program code to get the attributes item with dynamic access:



```
Copies the following program code to clipboard.

private static void
AccessingAttributesWithDynamicAccess(Device device)
{
    var attributeNames = new[]
    {
    "TypeName", "Author", "Comment"
    };
    foreach (var attributeName in attributeNames)
    {
    object attributeValue =
((IEngineeringObject)device).GetAttribute(attributeName);
    }
}
```

1.2 Get type identifier of devices and device items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal

- A project is open.

  See Opening a project

Introduction

The attribute `TypeIdentifier` is used to identify a hardware object that is created via TIA Portal Openness API. The `TypeIdentifier` is a string consisting of several parts: `<TypeIdentifierType>:<Identifier>`

Possible values for `TypeIdentifierType` are:

- `OrderNumber`
- `GSD`
- `System`

OrderNumber

`OrderNumber` is the common `TypeIdentifier` for all modules present in the hardware catalog.

Format of type identifier	Example	Specifics
`<OrderNumber>`	OrderNumber:3RK1 200-0CE00-0AA2	-
`<OrderNumber>/<FirmwareVersion>`	OrderNumber:6ES7 510-1DJ01-0AB0/V2.0	Firmware version is optional in case it does not exist or there is only one versionexisting in the system. Be care
`<OrderNumber>//<AdditionalTypeIdentifier>`	OrderNumber:6AV2 124-2DC01-0AX0//Landscape	The additional type identifier might be necessary in case that `OrderNumber` and `FirmwareVersion` do not lead to a unique match in the system.

Note

There are a few modules in the hardware catalog which use "wildcard" characters in the order number to represent a certain cluster of real hardware, e.g. the different lengths of S7-300 racks. In this case the specific `OrderNumber` and the "wildcard"-`OrderNumber` can both be used to create an instance of the hardware object. However you cannot generically use wildcards at any position.

GSD

This is the identifier used for modules that are added to the TIA Portal via GSD or GSDML.

Format of type identifier	Example	Specifics
`<GsdName>/<GsdType>`	GSD:SIEM8139.GSD/DAP	`GsdName` is the name of the GSD or GSDML in uppercase letters. `GsdType` is one of the following: • D: Device • R: Rack • DAP: HeadModule • M: Module • SM: Submodule GsdId is a identifier for the type.
`<GsdName>/<GsdType>/<GsdId>`	GSD:SIEM8139.GSD/M/4	

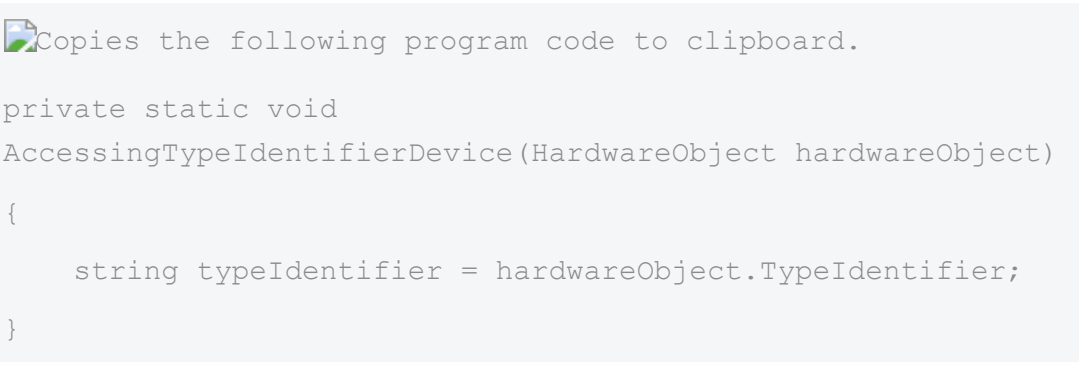
System

This is the identifier for objects, which cannot be determined using `OrderNumber` or `GSD`.

<SystemTypeIdentifier>/<AdditionalTypeIdentifier>	GSD:SIEM8139.GSD/M/4	• Subnet. • Device. • Rack.

Program code

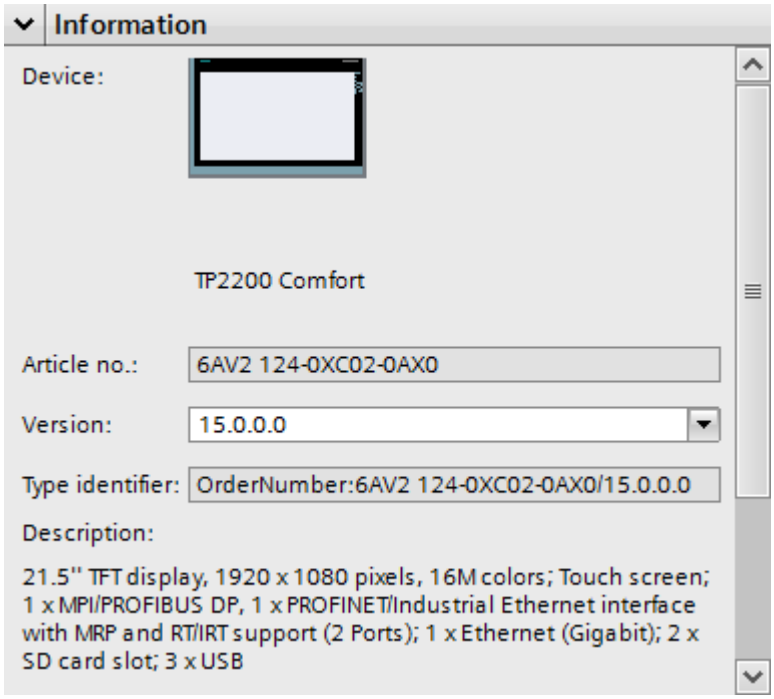
Modify the following program code to get the type identifier for user manageable and separately creatable objects for GSD:



Displaying type identifiers in TIA Portal

If you need to know a type identifier you inquire it in TIA Portal as follows:

1. Enable the setting "Enable display of the type identifier for devices and modules" in "Options > Settings > Hardware configuration > Display of the type identifier".
2. Open the editor "Devices & networks".
3. Select a device in the Catalog.
   The type identifier is displayed in the viewlet "Information"



1.3 Set App ID on device and device Items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to set the Application ID on device and device items of TIA Portal project, so that these Application IDs are retained in the current session of TIA Portal for later use.

You can set the Application ID by providing "Application Key" and "Application Value" pair to the API.

The following constraints applicable for the Application ID of device and device item object:

- An object of type supporting this feature can have maximum of 64 Application IDs available for it
- The Application IDs Key or/and Value length can be maximum 128 Characters in length.
- The Application ID Key for a given object is unique
- Only one Application ID can be set using one "Application Key"
- When different Application Value is set with the same "Application Key", the last set Application Value is retained

Program code

Modify the following program code to set the Application ID for device and device item object:

```
Copies the following program code to clipboard.

private static void SettingAppIDOnDevice(Device device)
{
    // Ask for Service CustomIdentityProvider on the
device/device item
    var customIdentityProviderService =
device.GetService<CustomIdentityProvider>();
    //Set the Application ID (Key-Value) pair
    customIdentityProviderService.Set("Application_Key",
"Application_Value");
}
```

1.4 Get App ID on device and device Items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project

Introduction

You can use the TIA Portal Openness to retrieve the Application ID of a device and device item in a TIA Portal project, provided they have already been assigned with Application IDs.

These IDs are stored in TIA Portal as key-value pair. To obtain the Application ID , You need to provide corresponding Application Key.

Program code

Modify the following program code to get the Application ID value for an object that is already using the Application Key:

```
Copies the following program code to clipboard.

private static void GettingAppIdOnDevice(Device device)
{
```

```
    // Ask for Service CustomIdentityProvider on the
device/device item
    var customIdentityProviderService =
device.GetService<CustomIdentityProvider>();
    customIdentityProviderService.Set("Application_Key",
"Application_Value");
    //Get the Application ID (Value) for the given
Application Key
    var applicationValue =
customIdentityProviderService.Get("Application_Key");
}
```

See also

Opening a project

1.5 Remove App ID on device and device items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to remove Application ID from device and device items in a TIA Portal project, allowing these objects to be updated with relevant Application ID.

A CustomIdentityNotFoundException will be thrown, if the provided Application ID does not exist.

Program code

Modify the following program code to remove the Application ID for device and device items:

```
 Copies the following program code to clipboard.
private static void RemoveApplicationIdOnDevice(Device
device)
{
```

```
    // Ask for Service CustomIdentityProvider on the
device/device item
    var customIdentityProviderService =
device.GetService<CustomIdentityProvider>();
    //Remove the CustomIdentity corresponding to Application
ID
    try
    {
    customIdentityProviderService.Remove("Application_Key");
    }
    catch(CustomIdentityNotFoundException ex)
    {
    // When CustomIdentity is not found for a the given key
"Application_Key".
    Console.WriteLine(ex.ToString());
    }
}
```

1.6 Creating a device

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

A device can be created via two methods, within a project or a device group:

- Create a device via a device item type identifier like in TIA hardware catalog
  `Device CreateWithItem(DeviceItemTypeId, DeviceItemName, DeviceName)`
- Create only the device
  `Device Create(DeviceTypeId, DeviceName)`

Name	Type	Description

DeviceItemTypeId	string	Type identifier of the device item
DeviceTypeId	string	Type identifier of the device
DeviceItemName	string	Name of the created device item
DeviceName	string	Name of the created device

See: Get type identifier of devices and device items

Program code

Modify the following code to create a device object via a type identifier:



```
Copies the following program code to clipboard.

private static void
CreateDeviceWithTypeIdentifier(DeviceComposition devices)
{
    Device device = devices.CreateWithItem
    ("OrderNumber:6ES7 510-1DJ01-0AB0/V2.0", "PLC_1",
"NewDevice");
    Device gsdDevice = devices.CreateWithItem
    ("GSD:SIEM8139.GSD/M/4 ", "GSD Module", "NewGsdDevice");
}
```

Program code

Modify the following code to create only the device object:



```
Copies the following program code to clipboard.

private static void DeviceObjectCreate(DeviceComposition
devices)
{
    Device deviceOnly = devices.Create("System:Device.S7300",
"S7300Device");
    Device gsdDeviceOnly =
devices.Create("GSD:SIEM8139.GSD/D", "GSD Device");
}
```

1.7 Enumerating devices

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

The TIA Portal Openness API positions devices similar to the project navigation in TIA Portal. PNV.

- Devices located as direct children of project are aggregated using the "Devices" composition of the project
- Devices located in device folders are aggregated using the "Devices" composition of the folder.

Note

Observe the Hierarchy of hardware objects of the object model.

Use one of the following options to enumerate the devices of a project:

- Enumerate all devices at root level
- Enumerate all devices in groups or sub-groups
- Enumerate all devices of a project that contains no device groups
- Enumerate all devices of the ungrouped device system groups

Examples of devices that can be enumerated:

- Central station
- PB-Slave / PN-IO device
- HMI Device

Program code

Modify the following program code to enumerate devices at root level:

```
Copies the following program code to clipboard.

private static void EnumerateDevicesInProject(Project
project)
{
    DeviceComposition deviceComposition = project.Devices;
```

```
    foreach (Device device in deviceComposition)
    {
    // add code here
    }
}
```

Modify the following program code to access an individual device.

```
Copies the following program code to clipboard.
private static void AccessSingleDeviceByName(Project project)
{
    DeviceComposition deviceComposition = project.Devices;
    // The parameter specifies the name of the device
    Device device = deviceComposition.Find("MyDevice");
}
```

Program code

To access devices in a group, you have to navigate to the group at first, after that to the device.

```
Copies the following program code to clipboard.
//Enamerate devices in groups or sub-groups
private static void EnumerateDevicesInGroups(Project project)
{
    foreach (DeviceUserGroup deviceUserGroup in
project.DeviceGroups)
    {
        EnumerateDeviceUserGroup(deviceUserGroup);
    }
}
private static void EnumerateDeviceUserGroup(DeviceUserGroup
deviceUserGroup)
{
    EnumerateDeviceObjects(deviceUserGroup.Devices);
```

```
    foreach (DeviceUserGroup subDeviceUserGroup in
deviceUserGroup.Groups)
    {
        // recursion
        EnumerateDeviceUserGroup(subDeviceUserGroup);
    }
}
private static void EnumerateDeviceObjects(DeviceComposition
deviceComposition)
{
    foreach (Device device in deviceComposition)
    {
    // add code here
    }
}
```

Program code

Modify the following program code to find a specific device by name:

```
Copies the following program code to clipboard.
private static void FindingSpecificDevice(Project project)
{
    //Find a specific device by name
    Device plc1 = project.Devices.First(d => d.Name ==
"Mydevice");
    // Work with the device
}
```

Modify the following program code to find a specific device via "Find" method:

```
Copies the following program code to clipboard.
private static void FindingSpecificDevice2(Project project)
{
    //Find a specific device via "Find" method
```

```
    Device plc1 = project.Devices.Find("MyDevice");
    // Work with the device
}
```

Program code

Modify the following program code to enumerate all devices which are located under a project that contains no device groups:



```
private static void EnumerateDevicesWithNoDeviceGroup(Project project)
{
    foreach (Device device in project.Devices)
    {
    // Work with the devices
    }
}
```

Program code

Modify the following program code to enumerate device located in a folder:



```
private static void EnumerateDeviceLocatedinFolder(Project project)
{
    DeviceUserGroup sortingGroup = project.DeviceGroups.Find("Sorting");
    Device plc1 = sortingGroup.Devices.First(d => d.Name == "MyStationName");
    // Work with the device
}
```

Program code

To structure the projects, decentral devices have been put to the UngroupedDevices group. To access this group, navigate to the group at first, after that to the device.

Modify the following program code to enumerate devices of the ungrouped device system groups:

```
Copies the following program code to clipboard.
private static void EnumerateUngroupedDevice(Project project)
{
    DeviceSystemGroup group = project.UngroupedDevicesGroup;
    Device plc1 = group.Devices.First(d => d.Name ==
"MyStationName");
    // Work with the device
}
```

1.8 Accessing devices

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

Every GSD or GSDML based IO device has attributes. Some of them are used to identify the specific type of the device.

Name	Data type	Writeable	Access	Description
Author	string	read/write	dynamic	-
Comment	string	read/write	dynamic	-
GsdName	string	read	dynamic	Name of the GSD or GSDML file.
GsdType	string	read	dynamic	Type of the hardware object. For devices the value is always "D".

GsdId	string	read	dynamic	Specific identifier for the hardware object. For devices always empty.
IsGsd	bool	read	modeled	TRUE in case of a GSD device or a GSDML device
Name	string	read/write	modeled	-
TypeIdentifier	string	read	modeled	-
GsdxContainerSignature	uint32	read	dynamic	This property returns the status of the digital signature (no signature available=0, valid signature=1, invalid signature=2).

Program code

Modify the following program code to get identification attributes:

```
Copies the following program code to clipboard.
private static void GetIdentificationAttribute(Device device)
{
    var attributeNames = new[]
    {
    "GsdName", "GsdType", "GsdId"
    };
    foreach (var attributeName in attributeNames)
    {
    object attributeValue =
device.GetAttribute(attributeName);
    }
}
```

Program code

Modify the following program code to get the attributes:

```
Copies the following program code to clipboard.
private static void GetAttribute(Device device)
{
    string nameValue = device.Name;
    bool isGsdValue = device.IsGsd;
}
```

Program code

Modify the following program code to get attribute with dynamic access:

```
Copies the following program code to clipboard.
private static void GetAttributeWithdDynamicAccess(Device
device)
{
    var attributeNames = new[]
    {
    "GsdName", "GsdType", "GsdId"
    };
    foreach (var attributeName in attributeNames)
    {
    object attributeValue =
device.GetAttribute(attributeName);
    }
}
```

Program code

If a device is a GSD device, it provides additional functionality. To get the GsdDevice feature, the GetService method is used.

```
Copies the following program code to clipboard.
private static void GetSpecificGSDDevice(DeviceItem
deviceItem)
{
```

```
    GsdDevice gsdDevice =
((IEngineeringServiceProvider)deviceItem).GetService<GsdDevice>
();
    if (gsdDevice != null)
    {
     // work with the GSD device
    }
}
```

Program code

Modify the following program code to get attributes of a GSD device

```
Copies the following program code to clipboard.
private static void GetAttributesGSDDevice(Device device)
{
    GsdDevice gsdDevice =
((IEngineeringServiceProvider)device).GetService<GsdDevice>
();
    string gsdId = gsdDevice.GsdId;
    string gsdName = gsdDevice.GsdName;
    string gsdType = gsdDevice.GsdType;
    bool isProfibus = gsdDevice.IsProfibus;
    bool isProfinet = gsdDevice.IsProfinet;
}
```

1.9 Accessing the TIA Portal hardware catalog

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can check manually required hardware in the catalog before running the Openness application. You can also use the TIA Portal Openns to respond correctly when a device is required to be added in the Openness application is not available in the hardware catalog for example - provide a message to the user that the GSD(ML) or HSP has to be installed first.

Attribute

A new type Siemens.Engineering.HW.HardwareCatalog.CatalogEntry is introduced which contains the following attribute members:

Attribute name	Data type	Description	Access
ArticleNumber	string	Specifies the article number of hardware object, for example. 6ES7 516-3AN00-0AB0	Read
CatalogPath	string	Specifies the catalog path of item for example: Root\PLC\SIMATIC S7-1500\CPU\CPU 1516-3 PN/DP\6ES7 516-3AN00-0AB0	Read
Description	string	Specifies the hardware catalog entry's description for example : • CPU with display; work memory 1 MB code and 5 MB data • 10 ns bit instruction time;	

		• 4-stage protection concept, integrated technology functions: Motion Control, closed-loop control, counting and measuring • integrated tracing • 1st interface: PROFINET IO controller, supports RT/IRT, 2 ports, MRP, transport protocol TCP/IP, S7 communication, Web server, constant bus cycle time, routing • 2nd interface: PROFINET basic services, transport protocol TCP/IP, Web server, routing • 3rd interface: PROFIBUS DP master, constant bus cycle time, routing; firmware V1.0	Read
TypeIdentifier	string	Specifies the type identifier of item. e.g.: OrderNumber:6ES7 516-3AN00-0AB0/V1.0	Read
TypeIdentifierNormalized	string	Specifies the normalized typeIdentifier of item. for example: OrderNumber:6ES7516-3AN00-0AB0/V1.0	Read
TypeName	string	Specifies the type name of the device for example.: CPU 1516-3 PN/DP	Read
Version	string	Specifies the hardware catalog entry's version. for example: V1.0	Read

Program code

Modify the following program code to get all hardware catalog items without any filter:

```
Copies the following program code to clipboard.
private static void GetAllAvailableHardwareCatalog()
{
    // Refer Connecting to the TIA Portal Openness section
    TiaPortal portal =
TiaPortal.GetProcesses().First().Attach();
    var itemList = portal.HardwareCatalog.Find(string.Empty);
```

```
}
```

Modify the following program code to to get hardware catalog entries by a filter string:

```
Copies the following program code to clipboard.
private static void GetHardwareCatalogEntryByFilter()
{
    TiaPortal portal =
TiaPortal.GetProcesses().First().Attach();
    var itemList = portal.HardwareCatalog.Find("1516");
    var first = itemList.First();
    Console.WriteLine(first.TypeIdentifier);
    //proj.Devices.Create("OrderNumber:6ES7 516-2GN00-
0AB0/V2.9", "PLC_1");
    Console.ReadKey();
}
```

Modify the following program code to get the hardware catalog items which are compatible with a specific TypeIdentifier and filter provided by string:

```
Copies the following program code to clipboard.
private static void FilterHardwareCatalogByTypeAndString()
{
    TiaPortal portal =
TiaPortal.GetProcesses().First().Attach();
    var itemList = portal.HardwareCatalog.Find("1BL00",
    "OrderNumber:6ES7 516-2GN00-0AB0/V2.9");
    // The itemList contains all the devices
    // which are compatible/pluggable with the given
TypeIdentifier
    // and filtered by the filter string.
    // In this case one of the itemList elements is 6ES7 521-
1BL00-0AB0/V1.0.
    Console.ReadKey();
}
```

1.10 Deleting a device

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code

Modify the following program code to delete a device:

```
Copies the following program code to clipboard.
private static void DeviceToDelete(Project project)
{
    // Needs to be: UngroupedDevicesGroup
    Device deviceToDelete =
project.UngroupedDevicesGroup.Devices.Find("......");
    // delete device
    deviceToDelete.Delete();
}
```

1.11 Bulk changing hardware parameters

Requirement

- The TIA Portal is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to configure multiple attributes of a hardware object using single SetAttributes call.

The API also removes the order and dependencies of the attributes and their corresponding values when function is called.

Signature

SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes, AttributeDelegate errorHandler);

Note

This is an overload of the SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes) API.
From the TIA Portal V19, Bulk changing hardware parameters with ordering works the same way with all overloaded SetAttributes methods.

Program code



```
Copies the following program code to clipboard.
private static void BulkChangeHardwareParameter(TiaPortal tiaPortal, Project project)
{
    string projectPath = "@ D:\\Openness\\Device_name";
    var project2 = tiaPortal.Projects.Open(new FileInfo(projectPath));
    IList<KeyValuePair<string, object>> attributesToSet = new List
    <KeyValuePair<string, object>>
    {
        new KeyValuePair<string, object>("IsochronousTi", (double)0.1),
        new KeyValuePair<string, object>("IsochronousTo", (double)0.3),
        new KeyValuePair<string, object>("IsochronousTiToCalculationMode",
        Siemens.Engineering.HW.IsochronousTiToCalculationMode.Manual),
        new KeyValuePair<string, object>("IsochronousMode", true)
    };
    project.SetAttributes(attributesToSet, attributeConfiguration =>
    {
    //this means, if one of the attribute set fails, the mechanism won't stop
    //just skip the problematic part and the other attributes will be set.
```

```
    attributeConfiguration.CurrentSelection =
AttributeChoiceSelection.Ignore;
    });
}
```

Functions on device items

2

2.1 Mandatory attributes of device items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

Every device or device item provides certain mandatory attributes which can be read and/or written. These attributes are always the same as in the TIA Portal user interface.

The following attributes are supported in TIA Portal Openness:

Attribute name	Data type	Writable	Access	Comment
Author	string	read/write	dynamic	-
Classification	DeviceItemClassification	read	modeled	The classification the device item belongs to, flags enum, see Device item classification description below.
Comment	string	read/write	dynamic	sometimes only read access
CommentML	MultilingualTextItem	read/write	dynamic	sometimes only read access

FirmwareVersion	string	read	dynamic	-
InterfaceOperatingMode	InterfaceOperatingModes	read/write	dynamic	For device items that provides the feature `NetworkInterface`
InterfaceType	NetType	readwrite	dynamic	For device items that provides the feature `NetworkInterface`
IsBuiltIn	bool	read	modeled	FALSE for objects creatable by the user
IsGsd	bool	read	modeled	TRUE, if the device description is installed via GSD/GSDML
IsPlugged	bool	read	modeled	TRUE for devices that are plugged
Label	string	read	dynamic	For device items that provides the feature `NetworkPort` or `NetworkInterface`. If the interface or port has no label `Label` will be `String.Empty`.
LocationIdentifier	string	read/write	dynamic	-
Name	string	read/write	modeled	sometimes only read access
OrderNumber	string	read	dynamic	-
PlantDesignation	string	read/write	dynamic	-
PositionNumber	int	read	modeled	-

TypeIdentifier	string	read	modeled	-
TypeName	string	read	dynamic	The language independent type name. Optional for device items that are not manageable by the user as auto-created items or fixed sub-modules).
InstallationDate	DateTime	read/write	dynamic	-
AdditionalInformation	string	read/write	dynamic	-

Device item classification

Value	Description
Siemens.Engineering.HW.DeviceItemClassifications.None	No classification.
Siemens.Engineering.HW.DeviceItemClassifications.CPU	The device item is a CPU
Siemens.Engineering.HW.DeviceItemClassifications.HM	The device item is a head module.

Program code: Mandatory attributes of a device item

Modify the following program code to get the mandatory attributes of a device item:



```
Copies the following program code to clipboard.

private static void
GetMandatoryAttributesDeviceItem(DeviceItem deviceItem)
{
    string nameValue = deviceItem.Name;
    string typeIdentfierValue = deviceItem.TypeIdentifier;
    int positionNumberValue = deviceItem.PositionNumber;
    bool isBuiltInValue = deviceItem.IsBuiltIn;
    bool isPluggedValue = deviceItem.IsPlugged;
}
```

Program code: Mandatory attributes with dynamic access

Modify the following program code to get the attributes item with dynamic access:

```
Copies the following program code to clipboard.

private static void
MandatoryAttributesWithDynamicAccess(DeviceItem deviceItem)
{
    var attributeNames = new[] {
    "TypeName", "Author", "Comment", "OrderNumber",
"FirmwareVersion",
    "PlantDesignation", "LocationIdentifier"
    };
    foreach (var attributeName in attributeNames)
    {
    object attributeValue = ((IEngineeringObject)deviceItem).
    GetAttribute(attributeName);
    }
    ((IEngineeringObject)deviceItem).SetAttribute("Comment",
"This is a comment.");
}
```

2.2 Creating and plugging a device item

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

The action `PlugNew(string typeIdentifier, string name, int positionNumber)` of `HardwareObject` is used to

- create a new device item and plug it into an existing hardware object
- create a new sub device item, e.g. a submodule, and plug it into a device item

If the action was succseful it returns the created device item object, otherwise a recoverable exception will be thrown.

By using the action `CanPlugNew(string typeIdentifier, string name, int positionNumber)` you can determine if creating and plugging is possible. If executing is not possible the action returns `false`.

If the method returns true, the action might still fail for the following unforeseen reasons.

- a position number is already taken by another device item
- the current device item cannot be plugged at the position although it is free
- the container does not provide the position number
- the name of the device item is already taken by an existing device item in the same container
- the device item cannot be plugged into the container
- the device is online

The following table shows the needed method parameters:

Name	Type	Description
`typeIdentifier`	string	type identifier of the created device item
`name`	string	name of created device item
`positionNumber`	int	position number of the created device item

Program code

Modify the following program code to plug a device item into an exisitng hardware object:

```
Copies the following program code to clipboard.
private static void PluggingDeviceItem
(HardwareObject hardwareObject, string typeIdentifier,
string name, int positionNumber)
{
    if(hwObject.CanPlugNew(typeIdentifier, name,
positionNumber))
    {
```

```
    Siemens.Engineering.HW.DeviceItem newPluggedDeviceItem =
    hwObject.PlugNew(typeIdentifier, name, positionNumber);
    }
}
```

Accessing module information

The TIA Portal Openness user can access information about the pluggable modules using ModuleInformationProvider object. The user can access

- the container types in which a specified module has to be plugged (ex. Device and Rack) using FindContainerTypes method.
- the available versions for a certain partly specified module using FindModuleTypes method.

Program code: Accessing ModuleInformationProvider object



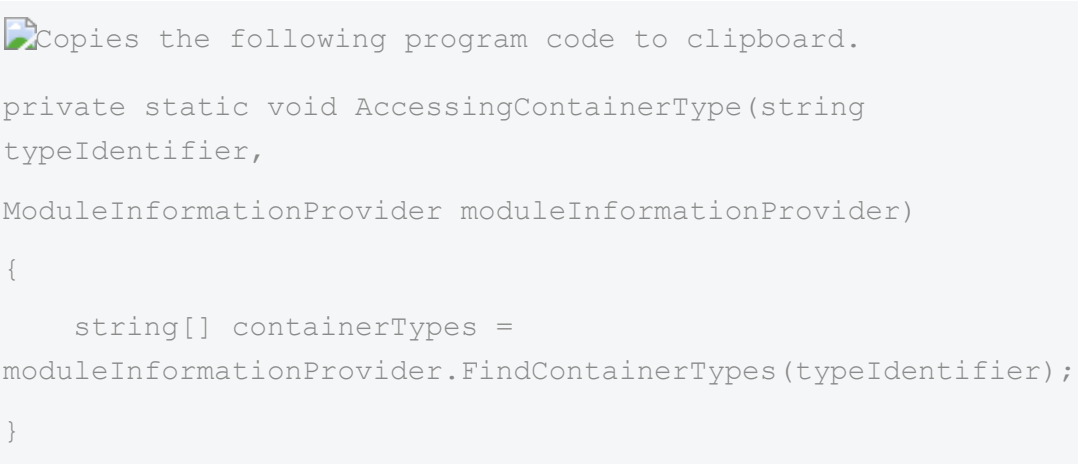
```
Copies the following program code to clipboard.
privaet static void ModuleInformationAccess(Project project)
{
    HardwareUtilityComposition extensions =
project.HwUtilities;
    var result = extensions.Find("ModuleInformationProvider")
    as ModuleInformationProvider;
}
```

Program code: Accessing container types using FindContinerTypes method

FindContinerTypes method returns the continer types of a given module. The module is speicified uisng typeIdentifier parameter. The resulting list contains the TypeIdentifiers of all container types of the requested type. Usually this includes a Device and a Rack and the containers are given in their order of the hierarchy in the project, beginning with the Device.

Name of the parameter	Type	Description
`typeIdentifier`	string	type identifier of a device item.

NOTICE
This method works only for the modules visible in the network view. This method works only for modules, and not for sub-modules.



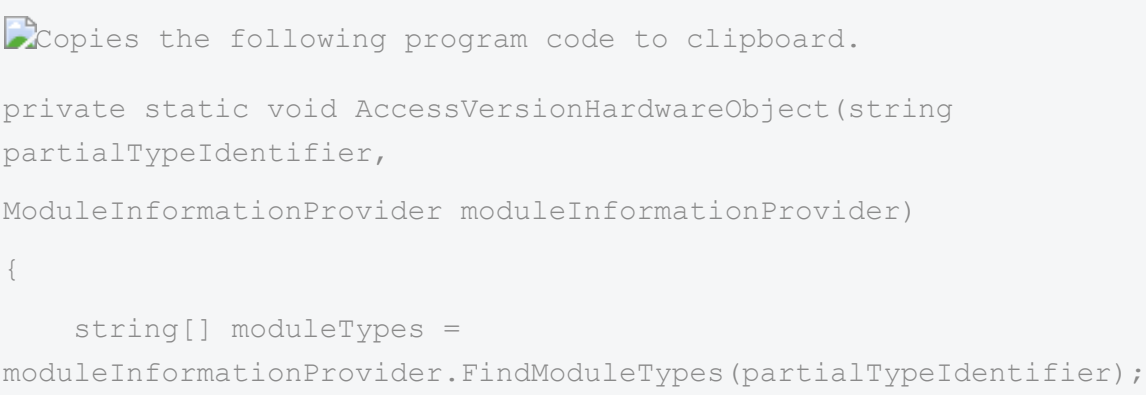
Program code: Accessing versions using FindModuleTypes method

FindModuleTypes method returns all possible versions of a hardware object using partial type identifier of the device item. This method returns a list of strings. Each string will be the complete TypeIdentifier of a possible match for the partial TypeIdentifier.

A valid partial type identifier must contain only complete parts, where each complete part is separated by "/" in the type identifier. Wildcards or incomplete parts are not supported. Also the following constraints with respect to the minimal number of specified parts must be observed:

- OrderNumber: at least one part. Ex. `OrderNumber:6ES7 317-2EK14-0AB0`
- GSD: at least two parts. Ex. `GSD:SI05816A.GSD/M`
- System: at lease one part. Ex. `System:Rack.ET200SP`

Name of the parameter	Type	Description
`partialTypeIdentifier`	string	partial type identifer of a device item



```
}
```

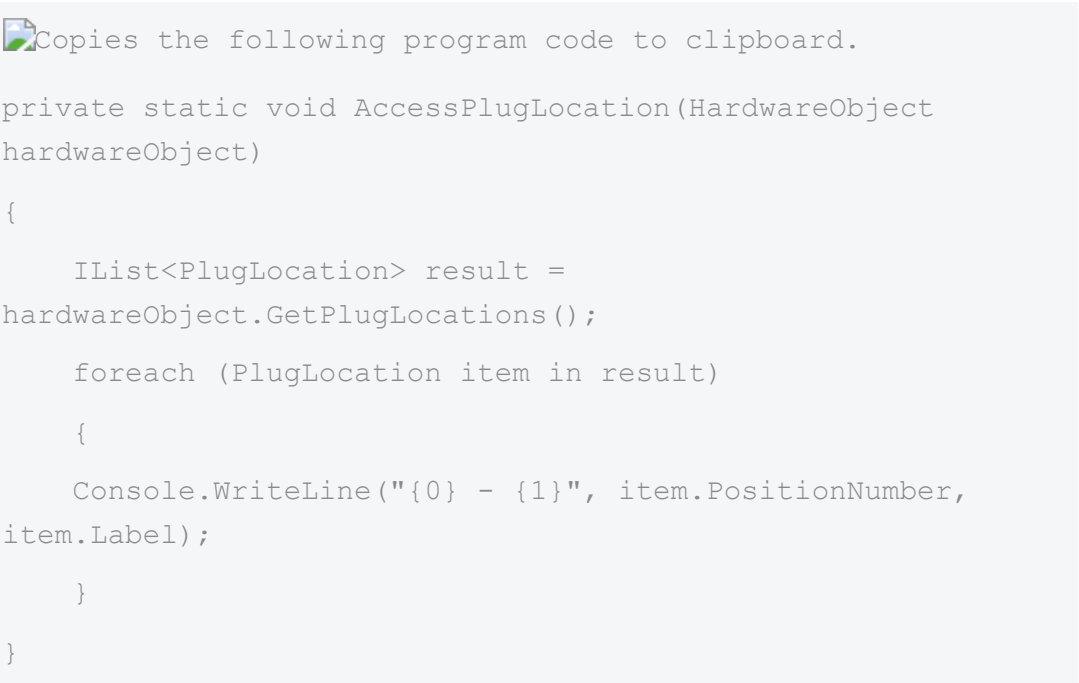
Program code: Accessing plug locations using GetPlugLocations method

GetPlugLocations method returns the information about slots such as plug location, position number (designation of a slot), and available slots for the hardware object.

The class PlugLocation has the following peroperites.

Name of the property	Type	Description
`PositionNumber`	int	The position number of the free slot
`Label`	string	The label of the free slot

- In case no "label" exists for a certain position number, the string representation of the position number is used.
- `PlugLocation` objects are provided only for free slots.



2.3 Moving device items into another slot

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal

- A project is open.

  See Opening a project

Introduction

The action PlugMove(DeviceItem deviceItem, int positionNumber) of HardwareObject is used to move an existing device item and plug it to an existing hardware object. The method PlugMove inserts the device items where the module was unable to plug in the UI. In these cases, there PlugMove action completes with complie errors.

The action CanPlugMove(DeviceItem deviceItem, int positionNumber) is used to determine possiblity of movement. If the movement is not possible, CanPlugMove returns false. If the method returns true, the action might still fail for the following unforeseen reasons.

- a position number is already taken by another device item
- the current device item cannot be plugged at the position although it is free
- the container does not provide the position number
- the name of the device item is already taken by an existing device item in the same container
- the device item cannot be plugged into the container
- the device item cannot be plugged by the user
- the device item cannot be removed by the user
- the device is online

Program code

Modify the following program code:



```
Copies the following program code to clipboard.
private static void
MoviceDeviceItemsIntoAnotherSlot(HardwareObject
hardwareObject,
DeviceItem deviceItemToMove, int positionNumber)
{
    if(hwObject.CanPlugMove(deviceItemToMove, positionNumber)
    {
    DeviceItem movedDeviceItem =
hwObject.PlugMove(deviceItemToMove, positionNumber);
    }
}
```

2.4 Copying a device item

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Use the action PlugCopy(DeviceItem deviceItem, int positionNumber) of HardwareObject to copy a device within a project and to plug it into an existing hardware. In rare cases the method PlugCopy might work where it is not possible to plug a module in the UI. In this case compile errors will occur after the copy. When PlugCopy was successful it returns the copy of the device item object, otherwise a recoverable exception is thrown.

Possible reasons for a failed action:

- a position number is already used by another device item
- the current device item cannot be plugged at the position although it is free
- the container does not provide the position number
- the name of the device item is already used by an existing device item in the same container
- the device item cannot be plugged into the container
- the device item cannot be plugged in the UI
- ...

Use the action CanPlugCopy(DeviceItem deviceItem, int positionNumber) is it possible to determine if copying should be possible. When it is not possible to execute the copy action CanPlugCopy returns false. However if the method returns true the action might still fail for unforeseen reasons.

Name of the parameter	Type	Description
deviceItem	DeviceItem	Device item to copy
positionNumber	int	position number for the copy of the device item

Program code

Modify the following program code:

```
Copies the following program code to clipboard.
HardwareObject hwObject = ...;
DeviceItem deviceItemToCopy = ...;
int positionNumber = ...;
if(hwObject.CanPlugCopy(deviceItemToCopy, positionNumber))
{
   DeviceItem copiedDeviceItem =
hwObject.PlugCopy(deviceItemToCopy, positionNumber);
}
```

2.5 Setting user defined logo settings

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A Project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to set the user defined logo settings functionality in the CPU display of the S71500 PLC. The functionality is not supported by all of the S71500 PLCs.

The feature FrontPanel Display is available at the CPU display deviceitem to provide the following method :

Method name	Data type	Description
SetUserDefinedLogo()	FileInfo	Set the path for the image to be uploaded in the CPU display. The SetUserDefinedLogo() can only be used if the if the UserDefinedLogoActivated property is enabled. Otherwise, an EngineeringTargetInvocation exception will be thrown if the UserDefinedLogoActivated property is disabled.

The follwing properties can only be available in the Openness if the CPU contains a display and the properties are also available in the TIA Portal.

Property name	Data type	Description
UserDefinedLogoActivated	boolean	Displays the user defined logo. To upload the image in the CPU display, the UserDefinedLogoActivated property should be enabled. It is not possible to upload the image, if the UserDefinedLogoActivated property is disabled. Otherwise, you have to reset the other settings after disabling the UserDefinedLogoActivated property.
AdaptLogoActivated	boolean	Displays the adapt logo. The AdaptLogoActivated property is only available if the UserDefinedLogoActivated is enabled.

Program code

```
using Siemens.Engineering.HW.Features.FrontPanelDisplay;
private void UserDefinedLogoSettings()
{
FrontPanelDisplay frontPanelDisplay =
plcDisplay.GetService<FrontPanelDisplay>();
//Enabling the user defined logo in the display
frontPanelDisplay.UserDefinedLogoActivated = true;
//Enabling the adapt logo in the display
frontPanelDisplay.AdaptLogoActivated = true;
//Uploading an image file in the display
frontPanelDisplay.SetUserDefinedLogo(new
FileInfo(@"C:\Pictures\CompanyLogo.jpg"));
}
```

2.6 Deleting a device item

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code

Modify the following program code to delete a device item:

```
Copies the following program code to clipboard.
Project project = ...;
var device =
project.UngroupedDevicesGroup.Devices.Find("......");
var deviceItem = deviceItem.DeviceItems.First();
// delete device item
deviceItem.Delete();
```

2.7 Enumerate device items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

To get to a device item use the HardwareObject. The items of a hardware objects are, what the user of the TIA Portal sees as being plugged into the hardware object:

- a rack which resides in a device
- a module which resides in a rack
- a sub module which resides in a module
- a sub module which resides in a sub module

Note

You can find more detailed information on this topic in the section Hierarchy of hardware objects of the object model.

Program code: Enumerating device items of a device

Modify the following program code to enumerate device items of a hardware object:



```
private static void EnumerateDeviceItems(HardwareObject
hardwareObject)
{
    foreach (DeviceItem deviceItem in hardwareObject.Items)
    {
        // add code here
    }
}
```

Program code: Enumerating with composition hierarchy

Modify the following program code if you want to enumerate the device items of a device by means of the composition hierarchy:



```
//Enumerates devices using an composition
private static void EnumerateDeviceItems(Device device)
{
    DeviceItemComposition deviceItemComposition =
device.DeviceItems;
    foreach (DeviceItem deviceItem in deviceItemComposition)
    {
        // add code here
    }
}
```

Program code: Enumerating devices items using an association

Modify the following program code to enumerate the device items using an association:

```
Copies the following program code to clipboard.
//Enumerates devices using an association
private static void
EnumerateDeviceItemsWithAssociation(Device device)
{
    DeviceItemAssociation deviceItemAssociation =
device.Items;
    foreach (DeviceItem deviceItem in deviceItemAssociation)
    {
        // add code here
    }
}
```

Program code: Accessing PLC module name within a project

Modify the following program code to access a specific PLC module within a project by name:

```
Copies the following program code to clipboard.
public static DeviceItem GetModuleByName(ProjectBase project)
{
    DeviceComposition devices = project.Devices;
    Device station = devices.Find("S71500/ET200MP
station_1");
    // The module name is unique in the logical relation via
    // "Items" within a container.
    DeviceItemAssociation stationModules = station.Items;
    DeviceItem rack = stationModules.Find("Rail_0");
    DeviceItemAssociation rackModules = rack.Items;
    DeviceItem plcModule = rackModules.Find("PLC_1");
    return plcModule;
}
```

Program code: Accessing PLC module by position within a project

Modify the following program code to access a specific PLC module within a project by its position:



```
Copies the following program code to clipboard.

public static DeviceItem
GetModuleByPositionNumber(ProjectBase project)
{
    DeviceComposition devices = project.Devices;
    Device station = devices.Find("S71500/ET200MP
station_1");
    // The position number is unique in the logical relation
via "Items" within a container.
    DeviceItemAssociation stationModules = station.Items;
    DeviceItem rack = stationModules.Find(0);
    DeviceItemAssociation rackModules = rack.Items;
    DeviceItem plcModule = rackModules.Find(1);
    return plcModule;
}
```

Program code: Accessing modules based on their classifications

Modiify the following program code to find a CPU based on their classification and return a list of device item:



```
Copies the following program code to clipboard.

public static IList<DeviceItem>
GetModulesByClassification(ProjectBase project)
{
    DeviceComposition devices = project.Devices;
    Device station = devices.Find("S7-1500R/H-System_1");
    // The classification is not a unique characteristic,
    // multiple items can have the same classification.
    // Find by classification is a search operation
    // that is provided both on "Items" and "DeviceItems".
```

```
    DeviceItemComposition stationModules =
station.DeviceItems;
    IList<DeviceItem> plcModules =
stationModules.Find(DeviceItemClassifications.CPU);
    return plcModules;
}
```

2.8 Accessing device items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

To access objects of the type "DeviceItem" use the following attributes:

- Name (string): the name of the device item
- Container (HardwareObject): the container into which the device item is plugged

Name	Data type	Writeable	Access	Description
Author	string	read/write	dynamic	-
Comment	string	read/write	dynamic	-
FirmwareVersion	string	read	dynamic	Only for head modules
GsdName	string	read	dynamic	Name of the GSD file.
GsdType	string	read	dynamic	Type of the hardware object. For devices the value is always "D".
GsdId	string	read	dynamic	Specific identifier for the hardware object. For devices always empty.
IsBuiltIn	bool	read	-	-

IsGsd	bool	read	modeled	TRUE in case of a GSD device or a GSDML device
IsPlugged	bool	read	modeled	-
IsProfibus	bool	read	modeled	-
IsProfinet	bool	read	modeled	-
Name	string	read/write	modeled	-
OrderNumber	string	read	dynamic	Only for head modules
PositionNumber	bool	read	modeled	-
TypeIdentifier	string	read	modeled	-

Program code: Accessing a device item

Modify the following program code to access a device item:



```
public static DeviceItem AccessDeviceItemFromDevice(Device device)
{
    DeviceItem deviceItem = device.DeviceItems[0];
    return deviceItem;
}
```

Program code: Accessing a device item of a device item

Modify the following program code to access a device item of a device item:



```
public static DeviceItem
AccessDeviceItemFromDeviceItem(DeviceItem deviceItem)
{
    DeviceItem subDeviceItem = deviceItem.DeviceItems[0];
    return subDeviceItem;
}
```

Program code: Navigating to the container of a device item

Modify the following program code to navigate back to the container of a device item via the "Container" attribute of DeviceItem:



```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
HardwareObject container = deviceItem.Container;
```

Program code: Get identification attributes

Modify the following program code to get the attributes:



```
Copies the following program code to clipboard.
Device device = ...;
var attributeNames = new[] {
    "GsdName", "GsdType", "GsdId" };
foreach (var attributeName in attributeNames) {
    object attributeValue =
((IEngineeringObject)deviceItem).GetAttribute(attributeName);
    }
```

Program code: Get attributes

Modify the following program code to get the attributes:



```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
GsdDeviceItem gsdDeviceItem =
((IEngineeringServiceProvider)deviceItem).GetService<GsdDeviceItem>
();
```

```
string gsdName = gsdDeviceItem.GsdName;
string gsdType = gsdDeviceItem.GsdType;
string gsdId = gsdDeviceItem.GsdId;
bool isProfinet = gsdDeviceItem.IsProfinet;
bool isProfibus = gsdDeviceItem.IsProfibus;;
```

Program code: Get attributes with dynamic access

Modify the following program code to get the attributes:



```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
GsdDeviceItem gsdDeviceItem =
((IEngineeringServiceProvider)deviceItem).GetService<GsdDeviceItem>
();
var attributeNames = new[] {
    "TypeName", "Author", "Comment", ...
    ;
foreach (var attributeName in attributeNames) {
    object attributeValue =
((IEngineeringObject)gsdDeviceItem).GetAttribute(attributeName);
    }
```

Program code: Set attributes

Modify the following program code to set the attributes:



```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
((IEngineeringObject)deviceItem).SetAttribute("Comment",
"This is a comment.");
```

Program code: Get prm data of a head module



```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
```

```
GsdDeviceItem gsdDeviceItem =
((IEngineeringServiceProvider)deviceItem).GetService<GsdDeviceItem>
();

int dsNumber = 0;       // For Profibus GSDs, dataset number zero
must be used!

int byteOffset = 0;

int lengthInBytes = 5;

// read complete data set:

byte[] prmDataComplete = gsdDeviceItem.GetPrmData(dsNumber,
byteOffset, lengthInBytes);

// read partial data set (only second byte):

byteOffset = 1;

lengthInBytes = 1;

byte[] prmDataPartial = gsdDeviceItem.GetPrmData(dsNumber,
byteOffset, lengthInBytes);
```

Program code: Set prm data of a head module

Modify the following program code to get the prm data:

```
 Copies the following program code to clipboard.

DeviceItem deviceItem = ...;

GsdDeviceItem gsdDeviceItem =
((IEngineeringServiceProvider)deviceItem).GetService<GsdDeviceItem>
();

// The parameters byteOffset and the length of the byte array
prmData define the range within the

// dataset which is written to.

// For Profibus GSDs, dataset number zero must be used!

// Change the highlighted bytes 2-4 from 0x0 to 0x1

// to write only the first two bytes: byte[] prmData = {0x05,
0x21};

int dsNumber = 0;

int byteOffset = 0;

byte[] prmData = {0x05, 0x21, 0x01, 0x01, 0x01};
```

```
gsdDeviceItem.SetPrmData(dsNumber, byteOffset, prmData);
```

Shared device assignment

You can configure IO devices and I-Devices in shared configurations via Openness.

The following attributes can be used to configure shared configuration in supported devices.

Name	Type	Description	Read/Write access
SharedDeviceAccessConfigured	bool	Defines whether a shareable (sub)module is assigned inside or outside the project.	R/W
SharedDeviceAssignmentTarget	DeviceItem	The controller the device item is assigned to (or null in case of outside of project assignment)	R/W
SharedDeviceAccessType	enumerated	Validate the shared status of the (sub)module	R
NumberOfExternalIControllers	uint	Defines the count of controllers, which have access to the current IO device, but are located in a different project.	R/W

The following values are available to the SharedDeviceAccessType attribute:

Value	Description
SharedDeviceAccessType.None	Fallback

SharedDeviceAccessType.Unknown	Inside the project but not assigned
SharedDeviceAccessType.OutsideOfProject	Assigned outside the project
SharedDeviceAccessType.PLC	Assigned inside the project to one of the connected PLCs

Note

These attributes are available only on device items which can be shared in a shared device context.You can use these attributes to detect such device items and proceed further with configuration.

See also

Hierarchy of hardware objects of the object model

2.9 Accessing device item as interface

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

If a device item is an interface, it provides additional functionalities over a simple device item. Using this interface, the user can access the nodes and operation mode of the interface. Due to this functionality, the device item can used as IoDevice (Secondary) or IoController (Primary) by accessing the NetworkInterface feature (a specific Service of the device item).

The properities of the interface is accessed using enum InterfaceOperatingModes.

Value	Description
InterfaceOperatingModes.None	Default
InterfaceOperatingModes.IoDevice	Interface operation mode "IoDevice" (Slave).
InterfaceOperatingModes.IoController	Interface operation mode "IoController" (Master).

InterfaceOperatingModes.IoDevice or InterfaceOperatingModes.IoController	Interface operation made both of the above.

Program code: Accessing the network interface feature

Modify the following program code to get the network interface feature

```
Copies the following program code to clipboard.
Device deviceItem = ...;
NetworkInterface itf =
((IEngineeringServiceProvider)deviceItem).GetService<NetworkInterface>
();
if (itf != null)
{
//Working with the interface
}
//Accessing nodes and operating mode
NodeComposition nodes = itf.Nodes;
InterfaceOperatingModes mode = itf.InterfaceOperatingMode;
//Accessing the type of interface
NetType itfType = itf.InterfaceType;
//Modififying the operating mode and interface type
itf.InterfaceOperatingMode = InterfaceOperatingModes.IoDevice;
itf.InterfaceType = NetType.Profibus;
//Accessing the ports linked to an interface.
NetworkPortAssociation nodes2 = itf.Ports;
```

2.10 Accessing change device

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the ChangeType() API to change the type of device or device item. There are some limitations to the feature in the TIA Portal Openness as compared to the TIA Portal user interface:

- No list of possible change partners provided.
- No feedback message available to inform about the type changes to the changed device or device item.

The modules supported in the TIA Portal user interface can also be used in the TIA Portal Openness. The following functionalities are not supported in the feature of the TIA Portal Openness:

- GSDML exchange
- Module description update

The ChangeType() API fails to change the type of the device or device item for the following reasons:

- The given TypeIdentifier is unknown.
- The object to be changed is read-only.
- The type change is not supported, such as when switching to a completely different module type or family.

Program code



```
Copies the following program code to clipboard.
using Siemens.Engineering.HW.DeviceItem;
private void ChangeTypeDeviceItem()
{
DeviceItem rack = ...;
DeviceItem deviceItem = rack.PlugNew("OrderNumber:6ES7 516-
3AN01-0AB0/V2.8", "PLC_1", 1);
try
{
deviceItem.ChangeType("OrderNumber:6ES7 516-3AN01-
0AB0/V2.9");
}
catch (EngineeringTargetInvocationException e)
{
```

```
Console.WriteLine("ChangeType failed: {0}", e.Message);
}
}
```

2.11 Accessing attributes of an I/O device interface

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- For writing access, the PLC is offline.

Application

You can use the TIA Portal Openness API interface to get or set attributes for IRT ans isochronous mode on the I/O device interface.

Access to the interface of an I/O controller

The following attributes can be accessed to the interface of an I/O controller. The controller has to be the sync master:

Attribute name	Data type	Writeable	Access	Description
`PnSendClock`	Int64	r/w	Dynamic attribute	Send clock in nanoseconds

Access to the interface of an I/O system

The following attributes can be accessed to the interface of an I/O system. The Ti/To values can be used by all modules and sub modules which belong to the I/O system.

Attribute name	Data type	Writeable	Access
`IsochronousTiToAutoCalculation`	BOOL	r/w	Dynamic attribute
`IsochronousTi`	DOUBLE	r/w	Dynamic attribute

IsochronousTo	DOUBLE	r/w	Dynamic attribute

Access to the interface of an I/O device

The following attributes can be accessed to the interface of an I/O device. The Ti/To values can be used by all modules and submodules which belong to the I/O sytem.

Attribute name	Data type	Writeable	Access
IsochronousMode	BOOL	r/w	Dynamic attribute
IsochronousTiToCalculationMode	IsochronousTiToCalculationMode	r/w	Dynamic attribute
IsochronousTi	DOUBLE	r/w	Dynamic attribute
IsochronousTo	DOUBLE	r/w	Dynamic attribute

The following ENUM values are provided for the attribute IsochronousTiToCalculationMode:

Value	Description
IsochronousTiToCalculationMode.None	-
IsochronousTiToCalculationMode.FromOB	Ti/To values of the OB (configured at the IoSystem) are used.
IsochronousTiToCalculationMode.FromSubnet	This value is not used by PROFINET interfaces.
IsochronousTiToCalculationMode.AutomaticMinimum	Ti/To values are calculated automatically for the IO Device.
IsochronousTiToCalculationMode.Manual	The user can enter Ti/To values for this IO Device manually.

Program code: Get or set attributes of an I/O device interface

Modify the following program code to access the send clock value:

```
Copies the following program code to clipboard.
DeviceItem pnInterface = ...;
// read attribute
long attributeValue =
(long)pnInterface.GetAttribute("PnSendClock");
// write attribute
long sendClock = 2000000;
pnInterface.SetAttribute("PnSendClock", sendClock);
```

Modify the following program code to access the Ti/To values of OB:

```
Copies the following program code to clipboard.
IoSystem ioSystem = ...;
bool titoAutoCalculation =
(bool)ioSystem.GetAttribute("IsochronousTiToAutoCalculation");
ioSystem.SetAttribute("IsochronousTiToAutoCalculation", true);
```

Modify the following program code to access the isochronous setting of an I/O device interface:

```
Copies the following program code to clipboard.
DeviceItem pnInterface = ...;
bool isochronousMode =
(bool)pnInterface.GetAttribute("IsochronousMode");
pnInterface.SetAttribute("IsochronousMode", true);
```

2.12 Getting subnet of device item

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the NetworkInterface service to access the navigator Nodes. Each node in turn provides access to the subnet link by calling the ConnectedSubnet property.

Program code

To get to the subnet of a device or device item modify the following program code:

```
Copies the following program code to clipboard.
using Siemens.Engineering.HW.Node;
private void GetSubnetDeviceItem()
{
DeviceItem itfDeviceItem = ...;
NetworkInterface itf =
itfDeviceItem.GetService<NetworkInterface>();
foreach (Node node in itf.Nodes)
{
Subnet subnet = node.ConnectedSubnet;
// work with the subnet
}
}
```

Note

In the above code the subnet is null if the node was not connected.

2.13 Accessing attributes of IoController

Requirements

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- For writing access, the PLC is offline.

Application

You can use the TIA Portal Openness API interface to get or set attributes for IoController. The following attributes are only available at PROFINET IoController (located below a Profinet interface). If the user can modify an attribute in UI, then the user can set the corresponding attribute through TIA Portal Openness.

Attribute name	Data type	Type	Access	Description
SyncRole	SyncRole	Read-write	Dynamic attribute	-
PnDeviceNumber	int	Read-only	Dynamic attribute	In TIA Portal UI, this property is located at the ethernet node (PROFINET section)

The Synchronication role property is available in PROFINET interface of the TIA Portal UI. The Enum SyncRole has the following values.

Enum value	Numerical value
SyncRole.NotSynchronized	0
SyncRole.SyncMaster	1
SyncRole.SyncSlave	2
SyncRole.RedundantSyncMaster	4

Program code: Setting attributes of IoController

```
Copies the following program code to clipboard.

IoController ioController= ...;

SyncRole syncRole = (SyncRole)
((IEngineeringObject)ioController).GetAttribute("SyncRole");

((IEngineeringObject)ioController).SetAttribute("SyncRole",
SyncRole.SyncMaster);
```

2.14 Accessing attributes of IoConnector

Requirements

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- For writing access, the PLC is offline.

Application

You can use the TIA Portal Openness API interface to get or set attributes for IoConnector. The following attributes are only available at PROFINET IoController (located below a Profinet interface). If the user can modify an attribute in UI, then the user can set the corresponding attribute through TIA Portal Oopenness.

There are four types of attributes such as update time attributes, watchdog time attributues, synchronization attributes and , device number attributes.

Update time attributes

The update time attributes are given below.

Attribute name	Data type	Type	Access	Description
PnUpdateTimeAutoCalculation	Boolean	Read-write	Dynamic attribute	If this attribute is true, then the update time is calculated automatically.
PnUpdateTime	Int64	Read-write	Dynamic attribute	Update time is measured in nano seconds.
PnUpdateTimeAdaption	Boolean	Read-write	Dynamic attribute	-

Watchdog time attributes

The watchdog time attributes are given below.

Attribute name	Data type	Type	Access	Description
PnWatchdogFactor	Int32	Read-write	Dynamic attribute	-

PnWatchdogTime	Int64	Read-only	Dynamic attribute	Watchdog time is measured in nano seconds.

Synchronization attributes

The synchronization attributes are given below.

Attribute name	Data type	Type	Access	Description
RtClass	RtClass	Read-write	Dynamic attribute	-
SyncRole	SyncRole	Read-only	Dynamic attribute	-

The Enum RtClass has following values.

Enum value	Numerical value
RtClass.None	0
RtClass.RT	1
RtClass.IRT	2

The Enum SyncRole has following values.

Enum value	Numerical value
SyncRole.NotSynchronized	0
SyncRole.SyncMaster	1
SyncRole.SyncSlave	2
SyncRole.RedundantSyncMaster	4

Device number attributes

The device number attributes are given below.

Attribute name	Data type	Type	Access	Description

PnDeviceNumber	int	Read-Write	Dynamic attribute	Indicates the device number.

Program code: Getting and setting attributes of IoConnector

```
Copies the following program code to clipboard.

IoConnector connector = ...
var attributeNames = new[] {
"PnUpdateTimeAutoCalculation", "PnUpdateTime",
"PnUpdateTimeAdaption", "PnWatchdogFactor", "PnWatchdogTime",
"RtClass", "SyncRole"
};
foreach (var attributeName in attributeNames)
 {
      object attributeValue = ((IEngineeringObject)connector
).GetAttribute(attributeName);
 }
connector.SetAttribute("PnUpdateTimeAutoCalculation", true);
```

See also

Opening a project

2.15 Accessing address controller

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

If a device item is an address controller, it provides additional functionality. To access the registered addresses of the address controller the role AddressController is used.

Program code: Get the address controller

Modify the following program code to get the address controller role:

```
Copies the following program code to clipboard.
AddressController addressController =
((IEngineeringServiceProvider)deviceItem).GetService<AddressController>
();
    if (addressController != null)
        {
            ... // work with the address controller
        }
```

Attributes of an address controller

The attributes of an address controller are:

- RegisteredAddresses

Modify the following program code to get the attributes of an address controller:

```
Copies the following program code to clipboard.
AddressController addressController = ...;
foreach (Address registeredAddress in
addressController.RegisteredAddresses)
        {
            ...
        }
```

2.16 Accessing addresses

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Address objects are aquired via the composition link `Addresses` of a device item. The attribute `Addresses` returns a collection of type `AddressComposition` which can be enumerated.

Program code: Get an address of a device item

To get the address of a device item modify the following program code:

```
Copies the following program code to clipboard.
AddressComposition addresses = deviceItem.Addresses;
    foreach(Address address in addresses)
        {
            // work with the address
        }
```

Program code: Get an address of an io controller

To get the address of an io controller modify the following program code:

```
Copies the following program code to clipboard.
AddressComposition addresses = ioController.Addresses;
    foreach(Address address in addresses)
        {
            // work with the address
        }
```

Attributes

An address supports the following attributes:

Attribute name	Data type	Writeable	Access	Comment
AddressControllers	AddressControllerAssociation	read	-	-
Context	enum: AddressContext	read	dynamic	only for diagnosis addresses an for special device items
IoType	enum: AddressIoType	read	-	-

StartAdress	Int32	read/write	modeled	-
Length	Int32	read	-	-

Value	Description
AddressIoType.Diagnosis	The type of the address io is Diagnosis.
AddressIoType.Input	The type of the address io is Input.
AddressIoType.Output	The type of the address io is Output.
AddressIoType.Substitute	The type of the address io is Substitute.
AddressIoType.None	The type of the address io is mot specified.

Value	Description
AddressContext.None	The address context is not applicable.
AddressContext.Device	A device address context.
AddressContext.Head	A head address context.

Program code: Read attributes

Modify the following program code to get the attributes:

```
 Copies the following program code to clipboard.

AddressControllerAssociation addressControllers =
address.AddressControllers;
Int32 startAddress = address.StartAddress;
AddressIoType addressType = address.IoType;
Int32 adressLength = address.Length;
```

Program code: Write attributes

Modify the following program code to write the attributes:

```
 Copies the following program code to clipboard.
```

```
Address addressControllers = ...;
address.StartAddress = intValueStartAddress;
```

Program code: Attributes with dynamic access

Modify the following program code to get the attributes:

```
Copies the following program code to clipboard.
Address address= ...;
object attributeValue =
((IEngineeringObject)address).GetAttribute("Context");
```

2.17 Accessing hardware identifiers

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

Hardware identifier objects are acquired from the following objects:

- Device
- DeviceItem
- IoSystem

The hardware identifier is represented by the class `HwIdentifier` and is accessed via the attribute `HwIdentifiers`.

Program code: Get the hardware identifier

To make HwIdentifier available modify the following program code:

```
Copies the following program code to clipboard.
var hwObject = ...;
foreach(HwIdentifier hardwareIdentifier in
hwObject.HwIdentifiers)
{
```

```
// Work with the HwIdentifer
}
```

Attributes of a hardware identifier

```
HwIdentifierControllerAssociation controllers =
hwIdentifier.HwIdentifierControllers;
Int64 Identifier = hwIdentifier.Identifier;
```

2.18 Accessing hardware identifier controller

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

If a device item is an hardware identifier controller, it is possible to access the registered hardware identifiers. To access these HwIdentifierController, a specific service of the device item, is used.

Program code: Get the hardware identifier controller

To get the HwIdentifierController modify the following program code:

```
HwIdentifierController hwIdentifierController =
((IEngineeringServiceProvider)deviceItem).GetService<HwIdentifierControl
();
    if (hwIdentifierController != null)
        {
            ... // work with the hardware identifier controller
        }
```

Program code: Attributes of a hardware identifier controller

The attributes of an address controller are:

- `RegisteredHwIdentifiers`: The hardware identifier controllers where the hardware identifier is registered.

Modify the following program code to get the attributes of an address controller:

```
Copies the following program code to clipboard.

HwIdentifierController hwIdentifierController = ...;

HwIdentifierAssociation controllers =
hwIdentifierController.RegisteredHwIdentifiers;
```

2.19 Accessing channels of device items

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal

- A project is open.

  See Opening a project

Introduction

A channel is represented by the Channel class. Channels are aquired from a device item via the attribute Channels of the DeviceItem class. The attribute Channels returns an implementation of ChannelComposition which can be enumerated. If the device items has no channels, the attribute Channels returns an empty collection.

Mandatory attributes

A channel supports the following mandatory attributes:

Attribute name	Data type	Writeable	Access	Comment
IoType	ChannelIoType	read	modelled	-
Type	ChannelType	read	modelled	-
Number	Int32	read	modelled	-
ChannelAddress	Int32	read	dynamic	Address of the channel in bits

ChannelWidth	UInt32	read	dynamic	Width of the channel in bits

Program code: Get channels of device item

Modify the following program code to get the channels of a device item:

```
Copies the following program code to clipboard.
ChannelComposition channels = deviceItem.Channels
foreach(Channel channel in channels)
    {
        // work with the channel
    }
```

Program code: Mandatory attributes of a channel

Modify the following program code to get the channels of a device item:

```
Copies the following program code to clipboard.
Channel channel = ...;
int channelNumber = channel.Number;
ChannelType type = channel.Type;
ChannelIoType ioType = channel.IoType;
```

Program code: Get values of attributes with dynamic access

Modify the following program code to get the values of dynamic attributes:

```
Copies the following program code to clipboard.
Channel channel = ...;
Int32 channelAddress = (Int32)
((IEngineeringObject)channel).GetAttribute("ChannelAddress");
UInt32 channelWidth = (UInt32)
((IEngineeringObject)channel).GetAttribute("ChannelWidth");
```

Program code: Set value of a dynamic attribute

Modify the following program code to set the value of a writeable dynamic attribute:

```
Copies the following program code to clipboard.
```

```
Channel channel = ...;
((IEngineeringObject)channel).SetAttribute("AnAttribute",
1234);
```

2.20 Accessing normalized type identifiers

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the GetTypeIdentifierNormalized() in the TIA Portal Openness to get the normalized format of TypeIdentifier of a module. The module is specified using a TypeIdentifier.

The GetTypeIdentifierNormalized() supports the following property:

Property name	Data type	Description
typeIdentifier	String	Specifies the type identifier of a device item

Program code




```
using Siemens.Engineering.HW.Utilities;
private void ProvideTypeIdentifier(ModuleInformationProvider
moduleInformationProvider)
{
string typeIdentifier = ...;
string normalizedTypeIdentifier =
moduleInformationProvider.GetTypeIdentifierNormalized(typeIdentifier);
}
```

2.21 Creating and exporting psc file

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- Opening a Project
  See Opening a project

Introduction

You can use the TIA Portal Openness application to create PSC file and download a device into it. To create a PSC file and download a device configuration into it, you can use the Export() method from the CardReaderPscProvider service, which is a part of the Siemens.Engineering.HW.Utilities namespace.

You can only create PSC file and download a device configuration into PSC file, if a PSC file does not exist in the parameter.

There is no separate command available to create and open a PSC file. If a PSC file exists as a parameter, the Export() method will throw an exception and will not overwrite the file.

Note

Because of safety reasons, the Export operation is not supported for f-activated devices (an exception will be thrown during export) in V18 and below.

From TIA Portal Openness v20 , you can export an encrypted PSC file. To export an encrypt PSC file, you can use overloaded Export() method with added Password parameter.

Note

The Psc encryption is supported on devices with CPU version V40.0 and higher.

Method

Method Name	Description
void Export(Device device, FileInfo fileName)	To export a PSC file and download a device configuration into PSC file. The Export() accepts the following parameters: • Device: Specifies the device to be exported • fileName : Specifies the filename that will be saved .psc
void Export(Device device, FileInfo fileName, SecureString password)	To export an encrypted PSC file. The Export() method is an method overloading that accepts the following additional parameter:

	• Password: To be used to encrypt or decrypt the archive's contents

Exception

The Siemens.Engineering.EngineeringTargetInvocationException will be thrown to you in following condition:

- If the selected device does not support encryption

The Siemens.Engineering.EngineeringPasswordPolicyViolationException will be thrown to you in following condition:

- If the given password is null
- If the given password does not match the policy criteria

Program code

To create and export a PSC file modify the following program code:

```
Copies the following program code to clipboard.
private static void PscFileExport(Project myProject)
{
    // Preconditions
    Device ipcDevice = myProject.Devices.Find("IPC427D");
    FileInfo exportFile = new FileInfo
    (@"C:\Users\Ertan\Documents\Automation\PC system
configuration74.psc");
    //Get the card reader provider from hardware utilities
    HardwareUtilityComposition utilities =
myProject.HwUtilities;
    HardwareUtility utility =
utilities.Find("CardReaderPscProvider");
    CardReaderPscProvider crp =
(CardReaderPscProvider)utility;
    //Perform the export
    crp.Export(ipcDevice, exportFile);
}
```

To export an encrypted PSC file modify the following program code:

```
Copies the following program code to clipboard.
```

```
private static void EncryptedPscFileExport(SecurePassword
password)
{
    Project testProject = project;
    HardwareUtilityComposition utilComp =
testProject.HwUtilities;
    HardwareUtility utility =
utilComp.Find("CardReaderPscProvider");
    CardReaderPscProvider cardReaderPscProvider =
(CardReaderPscProvider)utility;
    Device device =
testProject.Devices.Find("name_Of_Device");
    FileInfo fullPathOfFile = GetFullPathOfFile();
    SecureString password = GetSecurePassword();
    cardReaderPscProvider.Export(device, fullPathOfFile,
password);
}
```

2.22 Connection handling for extension racks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to fetch, add, and remove extension rack connections so that you can make use of TIA Portal Openness to implement extension rack connection support during CAx export/import.

Program code




```
ImConnection imConnection =
portDeviceItem.GetService<ImConnection>();
```

```
imConnection.Connect(partnerport);
imConnection.Disconnect();
imConnection.GetPartnerPort();
var imConnectionOwner = imConnection.OwnedBy;
```

See also

Connecting to the TIA Portal
Opening a project

2.23 Telecontrol management datapoint

Introduction

You can use the TIA Portal Openness application to export and import datapoints using CSV files. The TelecontrolManagement class, which extends the DeviceItem feature, facilitates the export and import actions for these datapoints.

Supporting Telecontrol Devices:

This feature is compatible with the following device versions:

- 6GK7 542-6VX00-0XE0 (V 2.3)
- 6GK7 542-6VX00-0XE0 (V 2.3)
- 6GK7 243-7SX30-0XE0 (V 3.5)
- 6GK7 243-1BX30-0XE0 (V 3.5)
- 6GK7 243-8RX30-0XE0 (V 3.5)
- 6GK7 242-7KX31-0XE0 (V 3.5)
- 6GK7 543-1MX00-0XE0 (V 2.4)

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- Opening a Project
  See Opening a project

Action

The TelecontrolManagement implements the following actions to manage datapoints:

Method name	Description

ExportDataPoints()	Generates a CSV file containing all relevant configuration data for the datapoints.
ImportDatapoints()	Replaces the currently configured datapoints of a device with those defined in a CSV file.

Program code

To export datapoint to CSV files modify the following program code:



```
Copies the following program code to clipboard.

private void ExportDataPointUsingCSV(DeviceItem cpDeviceItem)
{
    TelecontrolManagement telecontrolManagement =
cpDeviceItem.GetService<TelecontrolManagement>();
    bool
exportSuccessful
=
true
    try
    {
    telecontrolManagement.ExportDataPoints(new
FileInfo(csvFullFileName));
    }
    catch
(
EngineeringTargetInvocationException)
    {
    exportSuccessful
=
false;
    }
}
```

To import datapoint from CSV files modify the following program code:



```
Copies the following program code to clipboard.

private void ImportDataPointUsingCSV(DeviceItem cpDeviceItem)
```

```
{
    TelecontrolManagement telecontrolManagement =
cpDeviceItem.GetService<TelecontrolManagement>();
    bool
importSuccessful
=
true
;
    try
    {
    telecontrolManagement.ImportDataPoints(new
FileInfo(csvFullFileName));
    }
    catch
(EngineeringTargetInvocationException)
    {
    importSuccessful
=
false
;
    }
}
```

Telecontrol managemet datapoint functionalities

Navigator

```
public Siemens.Engineering.HW.TelecontrolDataPointComposition
TelecontrolDataPoints { get; }
```

The Siemens.Engineering.HW.TelecontrolDataPointComposition - Class provides already configured datapoints as a composition/ list for the user. It also forms the basis for datapoint mass operations or operations that are not tied to specific datapoints.



```
Copies the following program code to clipboard.

TelecontrolDataPointComposition telecontrolDataPoints =
telecontrolManagement.TelecontrolDataPoints;
foreach (TelecontrolDataPoint telecontrolDataPoint in
telecontrolDataPoints)
{
```

```
// Do something with the telecontrolDataPoint
}
```

TelecontrolDataPointComposition Actions

Action	Description
`public T Create<T> (Siemens.Engineering.IEngineeringObject tag)`	Datapoints can be created using the function provided by the TelecontrolDataPointComposition. **Generic Types - T:** The valid datapoint type T depends on the protocol type configured on the telecontrol device. For each configurable protocol on the telecontrol device, there is a corresponding datapoint class. These protocol-specific datapoint classes are derived from an abstract base class. Valid Generic Types: • Siemens.Engineering.HW.TelecontrolDnp3DataPoin • Siemens.Engineering.HW.TelecontrollecDataPoint • Siemens.Engineering.HW.TelecontrolSt7DataPoint • Siemens.Engineering.HW.TelecontrolWdcDataPoint **Parameter - IEngineeringObject tag:** Valid Parameter types: • Siemens.Engineering.SW.Tags.PlcTag • Siemens.Engineering.SW.Blocks.Interface.Member • null - reference

Code example for datapoint creation based on a telecontrol device with Dnp3 - protocol configuration

```
Copies the following program code to clipboard.

// Creates a new TelecontrolDnp3DataPoint without a valid
reference to any plc or db tag of the plc
IEngineeringObject anyTag = null;
TelecontrolDnp3DataPoint createdDnp3DataPoint =
telecontrolDataPoints.Create<TelecontrolDnp3DataPoint>
(anyTag);
// Creates a new TelecontrolDnp3DataPoint with a valid
reference to the passed plcTag of the plc
PlcTag plcTag = ...;
createdDnp3DataPoint =
```

```
telecontrolDataPoints.Create<TelecontrolDnp3DataPoint>
(plcTag);
// Creates a new TelecontrolDnp3DataPoint with a valid
reference to the passed dbTag of the plc
Member dbTag = ...;
createdDnp3DataPoint =
telecontrolDataPoints.Create<TelecontrolDnp3DataPoint>
(dbTag);
```

Datapoints

This abstract TelecontrolDataPoint base-class provides protocol-independent properties and functions. All protocol-specific datapoint-classes (TelecontrolDnp3DataPoint, TelecontrolIecDataPoint, TelecontrolSt7DataPoint, TelecontrolWdcDataPoint) are derived from this abstract base-class.

Protocol-specific member are provided by the derived classes.

- TelecontrolDataPoint -Description: Respresents an abstract datapoint
- TelecontrolDnp3DataPoint-Description: Respresents a Dnp3 protocol datapoint
- TelecontrolIecDataPoint-Description: Respresents an Iec protocol datapoint
- TelecontrolSt7DataPoint-Description: Respresents a St7 protocol datapoint
- TelecontrolWdcDataPoint-Description: Respresents a Telecontrol Basic (Wdc) protocol datapoint

Attributes/Properties

Certain attributes can only be accessed via dynamic attribute access. Properties can also be accessed via dynamic attribute access.

Attribute	Description
`public string Name { get; set; }`	Defines the unique name of the datapoint on the telecontrol device Available in all five classes.
`public byte DataPointType { get; set; }`	Defines the protocol-specific datapoint type Available in all five classes.
ReadCycle	The dynamic ReadCycle-Attribute provides read and write access to the datapoint read cycle and defines the priority of cyclic reading values of input datapoints from their assigned CPU tags. Possible values for the dynamic ReadCycle-Attribute are provided by the DataPointReadCycle-Enumeration: Possible values for the read-cycle of each telecontrol datapoint.

	Available in all five classes.
`public UInt32 DataPointIndex { get; set; }`	The DataPointIndex-Property provides read and write access to the datapoint index. Defines the datapoint index to address the datapoint. Not available in TelecontrolDataPoint and TelecontrolSt7DataPoint-
`public bool MasterFunction { get; set; }`	The MasterFunction-Property provides read and write access to the datapoints master function.Defines whether the value of the datapoint are sent as for a master. Available in TelecontrolDnp3DataPoint and TelecontrollecDataPoint

Attribute/Value - Validation

Dynamic Attributes

Attribute has READ - State: The status implies that the attribute has no effect in this context. Writing and Reading access of the attribute is not supported and throws an EngineeringNotSupportedException.



```
Copies the following program code to clipboard.

Assert.Throws<EngineeringNotSupportedException>(() =>
validDataPoint.GetAttribute(attributeName));
Assert.Throws<EngineeringNotSupportedException>(() =>
validDataPoint.SetAttribute(attributeName, anyValue));
```

Attribute has READ_ONLY - State: The status implies that the attribute is valid in this context but is not enabled for writing access. Reading access to the attribute is supported.

```
var currentValue = validDataPoint.GetAttribute(attributeName);
```

Writing access to the attribute is not supported and throws an EngineeringNotSupportedException.

```
Assert.Throws(() => validDataPoint.SetAttribute(attributeName,
anyValue));
```

Actions

Action	Description
`public void Delete()`	Datapoints can be deleted using the following function provided by the TelecontrolDataPoint and all derived datapoint classes.

Example - datapoint deletion:



```
Copies the following program code to clipboard.

// Deletion of a TelecontrolDnp3DataPoint Instance
TelecontrolDnp3DataPoint dnp3DataPoint = ...;
dnp3DataPoint.Delete();
// The already retrieved dnp3DataPoint Instance and the
TelecontrolDataPointComposition Instance is invalid from now
on.

// To obtain the composition of all valid datapoints it must
be reloaded.
// Deletion of a generic TelecontrolDataPoint Instance
TelecontrolDataPoint genericDataPoint = dnp3DataPoint;
genericDataPoint.Delete();
```